

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (Bsc.IT Semester III)
Data Structures Practical

Practical - V

Roll No. : S056	Name: Shweta.Mohan.Singh
Class : SYIT	Batch : B1
Date of Assignment : 21/7/26	Date/Time of Submission : 21/7/26

Binary Search Tree: Write a program to:

- a. Create a binary search tree.

Code:-

```
1 class TreeNode:
2     def __init__(self, data):
3         self.data = data
4         self.left = None
5         self.right = None
6 root = TreeNode('R')
7 nodeA = TreeNode('A')
8 nodeB = TreeNode('B')
9 nodeC = TreeNode('C')
10 nodeD = TreeNode('D')
11 nodeE = TreeNode('E')
12 nodeF = TreeNode('F')
13 nodeG = TreeNode('G')
14
15 root.left = nodeA
16 root.right = nodeB
17 nodeA.left = nodeC
18 nodeA.right = nodeD
19 nodeB.left = nodeE
20 nodeB.right = nodeF
21 nodeF.left = nodeG
22 print("root.right.left.data:", root
      .right.left.data)
23 print("Shweta,S056")
```

Output:-

```
root.right.left.data: E  
Shweta,S056
```

```
=== Code Execution Successful ===
```

- b. Insert nodes into a binary search tree.

Code:-

```
1 class TreeNode:
2     def __init__(self,data):
3         self.data=data
4         self.left=None
5         self.right=None
6 root=TreeNode('R')
7 nodeA=TreeNode('A')
8 nodeB=TreeNode('B')
9 nodeC=TreeNode('C')
10 nodeD=TreeNode('D')
11 nodeE=TreeNode('E')
12 nodeF=TreeNode('F')
13 nodeG=TreeNode('G')
14 nodeH=TreeNode('H')
15 root.left=nodeA
16 root.right=nodeB
17 nodeA.left=nodeC
18 nodeA.right=nodeD
19 nodeB.left=nodeE
20 nodeB.right=nodeF
21 nodeF.left=nodeG
22 nodeF.right=nodeH
23 print("root.right.right.right.data:"
24       ,root.right.right.right.data)
25 print("Shweta,S056")
```

Output:-

```
root.right.right.right.data: H  
Shweta,S056
```

- c. Search for a node in a binary search tree.

Code:-

```
class TreeNode:
```

```
    def __init__(self, data):
```

```
        self.data = data
```

```
        self.left = None
```

```
        self.right = None
```

```
root = TreeNode('R')
```

```
nodeA = TreeNode('A')
```

```
nodeB = TreeNode('B')
```

```
nodeC = TreeNode('C')
```

```
nodeD = TreeNode('D')
```

```
nodeE = TreeNode('E')
```

```
nodeF = TreeNode('F')
```

```
nodeG = TreeNode('G')
```

```
nodeH = TreeNode('H')
```

```
root.left = nodeA
```

```
root.right = nodeB
```

```
nodeA.left = nodeC
```

```
nodeA.right = nodeD
```

```
nodeB.left = nodeE
```

```
nodeB.right = nodeF
```

```
nodeF.left = nodeG
```

```
nodeF.right = nodeH
```

```
def search(root, key):
```

```
    if root is None:
```

```
        return False
```

```
    print("Checking:", root.data)
```

```
    if root.data == key:
```

```
        return True
```

```
    return search(root.left, key) or search(root.right, key)
```

```
key = input("Enter element to search: ")
```

```
if search(root, key):
```

```
    print("Element Found")
```

```
else:
```

```
    print("Element Not Found")
```

Output:-

Enter element to search: G

Checking: R

Checking: A

Checking: C

Checking: D

Checking: B

Checking: E

Checking: F

Checking: G

Element Found

Curiosity Questions:

1. In a BST, which side (left or right) has **smaller numbers** than the root?
2. If we insert numbers 10, 20, 5 into a BST, which number will be at the root?
3. What will the inorder traversal of a BST give — sorted numbers or random numbers?
4. Where will the **largest number** in a BST always be found?
5. If the root of a BST is 50 and we search for 70, will we go left or right? Why?
6. If a BST has only **one node**, what is its inorder traversal?
7. Can we create a BST from characters like A, B, C instead of numbers?
8. If we insert numbers 5, 10, 15, 20 in order, will the tree be balanced or like a straight line?
9. In a BST, do we check the left child first or the right child first in inorder traversal?
10. If we delete the root of a BST, will the tree disappear?